

Task 5

Time Series Preprocessing & Forecasting

This task continues directly from **Task 4**, where you pulled hourly/daily weather data from a public API and performed exploratory data analysis (trend, seasonality, missing-value checks, distribution plots). In Task 5 you will turn that cleaned dataset into a proper **forecasting pipeline**: preprocess it correctly for time series, build and evaluate an **ARIMA/SARIMA** model, and — as a bonus — compare it against a deep learning approach (**RNN/LSTM**).

Learning Objectives

- Prepare a raw time series for modeling: resampling, missing-value handling, and feature engineering.
- Understand and test for **stationarity** (ADF / KPSS) and apply differencing when required.
- Select ARIMA/SARIMA orders using ACF/PACF plots and automated search (`auto_arima`).
- Fit, diagnose, and forecast with ARIMA/SARIMA, including confidence intervals.
- Evaluate forecasts with the correct metrics and a **time-aware** validation strategy.
- (Bonus) Build an LSTM/RNN forecaster and compare it against the statistical baseline.

1. Recap: Where Task 4 Left Off

By the end of Task 4 you should have a DataFrame indexed by timestamp with at least one target weather variable (e.g. temperature), pulled from an API such as Open-Meteo, OpenWeatherMap, or Visual Crossing, along with your EDA notebook covering:

- Missing timestamp / gap detection in the raw pull.
- Trend and seasonality visual inspection (daily and yearly cycles).
- Distribution plots (histograms, boxplots) and outlier spotting.
- Correlation between weather variables (temperature, humidity, pressure, wind speed).

TIP — Before you continue

Confirm your index is a proper `DatetimeIndex` with a defined frequency (`df.index.freq`). Everything downstream — resampling, ARIMA seasonal order, LSTM windowing — depends on this being correct.

2. Preprocessing for Time Series

2.1 Resampling & Frequency Alignment

Weather APIs sometimes return irregular or mixed-frequency data. Resample to a fixed frequency (hourly or daily, matching your forecast horizon) before anything else.

```
df = df.set_index('timestamp').sort_index()
df = df.asfreq('H')           # or 'D' for daily
print(df.isna().sum())       # gaps introduced by resampling
```

2.2 Handling Missing Values

Weather variables are smooth and autocorrelated, so time-aware interpolation is preferred over mean/median imputation.

```
df['temperature'] = df['temperature'].interpolate(method='time', limit_direction='both')
df['humidity'] = df['humidity'].interpolate(method='linear', limit=6)
```

- **Short gaps ($\leq$ a few hours):** linear/time interpolation.
- **Longer gaps:** seasonal-naive fill (value from same hour, previous day/week) rather than a flat interpolation.
- Never forward-fill long stretches for a continuous variable like temperature — it destroys the signal.

2.3 Outlier Treatment

- Flag physically impossible values (e.g. negative humidity, temperature outside plausible range for the region).
- Use rolling z-score or IQR on a rolling window (not the global one) since weather is seasonal.
- Prefer capping/winsorizing over deletion to keep the series continuous for ARIMA.

2.4 Feature Engineering

- **Calendar features:** hour of day, day of week, month, is_weekend (mainly useful for the LSTM/ML side, not classic ARIMA).
- **Lag features:** t-1, t-24 (previous day same hour), t-168 (previous week).
- **Rolling statistics:** rolling mean/std over 24h and 7d windows to capture local trend.
- **Cyclical encoding** for hour/month using sine/cosine so the model sees continuity (hour 23 next to hour 0).

```
df['hour_sin'] = np.sin(2*np.pi*df.index.hour/24)
df['hour_cos'] = np.cos(2*np.pi*df.index.hour/24)
df['lag_24']   = df['temperature'].shift(24)
df['roll_mean_24'] = df['temperature'].rolling(24).mean()
```

2.5 Train/Test Split — Time-Aware Only

TIP — Critical rule

Never shuffle a time series split. Use a chronological cutoff (e.g. last 2–4 weeks as test) so the model is evaluated the way it will actually be used: forecasting the future from the past. Shuffling causes leakage and inflated accuracy.

```
split_date = df.index.max() - pd.Timedelta(days=21)
train, test = df.loc[:split_date], df.loc[split_date:]
```

2.6 Scaling

- **ARIMA/SARIMA:** no scaling needed — it models the raw (or differenced) series directly.
- **RNN/LSTM:** scale with `MinMaxScaler` or `StandardScaler`, fit **ONLY** on the training split, then transform train and test with that same fitted scaler.

3. Stationarity Checks

ARIMA assumes (after differencing) a stationary series — constant mean/variance and autocorrelation structure over time. Test before you model.

Test	Null Hypothesis (H0)	Interpretation
ADF (Augmented Dickey-Fuller)	Series has a unit root (non-stationary)	p-value < 0.05 → reject H0 → stationary
KPSS	Series is stationary	p-value < 0.05 → reject H0 → non-stationary

```
from statsmodels.tsa.stattools import adfuller, kpss

adf_stat, adf_p, *_ = adfuller(train['temperature'].dropna())
kpss_stat, kpss_p, *_ = kpss(train['temperature'].dropna(), regression='c')
print(f'ADF p-value: {adf_p:.4f} | KPSS p-value: {kpss_p:.4f}')
```

TIP — Running ADF and KPSS together

Using both avoids relying on a single test. If they disagree, it often signals the series is trend-stationary or seasonally non-stationary rather than a simple unit-root case — inspect the seasonal decomposition (`seasonal_decompose`) before deciding on 'd' and 'D'.

Differencing

```
df['temp_diff1'] = df['temperature'].diff(1)          # first difference (trend)
df['temp_diff_seasonal'] = df['temperature'].diff(24) # seasonal difference (daily cycle)
```

Weather data typically has a strong **daily seasonal cycle** (period 24 for hourly data) and often a weaker yearly one. This is exactly the case SARIMA is designed for — plain ARIMA will under-fit the seasonal pattern.

4. ARIMA / SARIMA Modeling

4.1 Choosing Orders with ACF / PACF

- **ACF (autocorrelation):** tails off gradually → suggests an AR component; sharp cutoff after lag q → suggests MA(q).
- **PACF (partial autocorrelation):** sharp cutoff after lag p → suggests AR(p).
- Look for a spike at lag 24 (or your seasonal period) in both plots — that's your seasonal signal.

```

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(train['temp_diff1'].dropna(), lags=48, ax=axes[0])
plot_pacf(train['temp_diff1'].dropna(), lags=48, ax=axes[1])
plt.tight_layout(); plt.savefig('acf_pacf.png', dpi=150)

```

4.2 Automated Order Search

```

import pmdarima as pm

auto_model = pm.auto_arima(
    train['temperature'],
    seasonal=True, m=24,                # daily seasonality for hourly data
    stepwise=True, trace=True,
    suppress_warnings=True,
    error_action='ignore'
)
print(auto_model.summary())

```

TIP — auto_arima vs manual selection

Have trainees do BOTH: read ACF/PACF manually to propose (p,d,q)(P,D,Q,m), then run `auto_arima` and compare. This builds intuition instead of treating the search as a black box — and mismatches are usually informative.

4.3 Fitting SARIMA

```

from statsmodels.tsa.statespace.sarimax import SARIMAX

model = SARIMAX(
    train['temperature'],
    order=(2, 1, 2),
    seasonal_order=(1, 1, 1, 24),
    enforce_stationarity=False,
    enforce_invertibility=False
)
fit = model.fit(dispatch=False)
print(fit.summary())

```

4.4 Residual Diagnostics

- `fit.plot_diagnostics(figsize=(10,8))` — check standardized residuals, histogram vs normal, Q-Q plot, and correlogram.
- Residuals should look like white noise: no remaining autocorrelation, roughly zero mean.
- Run a **Ljung-Box test** on residuals — a high p-value (> 0.05) supports 'no significant autocorrelation left'.

```
from statsmodels.stats.diagnostic import acorr_ljungbox
lb = acorr_ljungbox(fit.resid.dropna(), lags=[24], return_df=True)
print(lb)
```

4.5 Forecasting with Confidence Intervals

```
forecast = fit.get_forecast(steps=len(test))
pred_mean = forecast.predicted_mean
ci = forecast.conf_int(alpha=0.05)

plt.figure(figsize=(11,4))
plt.plot(train.index[-200:], train['temperature'][-200:], label='train')
plt.plot(test.index, test['temperature'], label='actual')
plt.plot(test.index, pred_mean, label='forecast', color='crimson')
plt.fill_between(test.index, ci.iloc[:,0], ci.iloc[:,1], color='crimson', alpha=0.15)
plt.legend(); plt.title('SARIMA Forecast vs Actual')
plt.savefig('sarima_forecast.png', dpi=150)
```

5. Evaluation

5.1 Metrics

Metric	Formula (intuition)	Notes
MAE	$\text{mean}(\text{actual} - \text{predicted})$	Same units as target; robust, easy to explain.
RMSE	$\sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$	Penalizes large errors more; sensitive to spikes.
MAPE	$\text{mean}(\text{actual} - \text{predicted} / \text{actual}) \times 100$	Unstable if actual values near zero (careful with temp in °C).
sMAPE	symmetric version of MAPE	More stable near-zero than MAPE.
AIC / BIC	model fit penalized by complexity	Use to compare ARIMA order candidates, not across model families.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(test['temperature'], pred_mean)
rmse = np.sqrt(mean_squared_error(test['temperature'], pred_mean))
mape = np.mean(np.abs((test['temperature'] - pred_mean) / test['temperature'])) * 100
print(f'MAE={mae:.2f} | RMSE={rmse:.2f} | MAPE={mape:.2f}%')
```

5.2 Time-Aware Validation: Walk-Forward / Rolling Origin

A single train/test split gives one noisy estimate. Use rolling-origin (walk-forward) validation: refit or update the model at each step, forecast a short horizon, slide the origin forward, repeat.

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5, test_size=24) # 5 rolling 24h test windows
scores = []
for train_idx, test_idx in tscv.split(df):
    tr, te = df.iloc[train_idx], df.iloc[test_idx]
    m = SARIMAX(tr['temperature'], order=(2,1,2), seasonal_order=(1,1,1,24)).fit(dispatch=False)
    pred = m.get_forecast(steps=len(te)).predicted_mean
    scores.append(mean_absolute_error(te['temperature'], pred))
print('Mean MAE across folds:', np.mean(scores))
```

5.3 Suggested Plots

- Actual vs forecast with shaded confidence interval (above).
- Residuals over time — check for remaining pattern or heteroscedasticity.
- Residual histogram + Q-Q plot — check near-normality.
- Error by forecast horizon (MAE at t+1, t+2, ... t+24) — shows how fast accuracy degrades.
- Model comparison bar chart (MAE/RMSE across ARIMA, SARIMA, and the bonus LSTM).

6. Bonus: RNN & LSTM Forecasting

BONUS — This section is optional

Not required to pass Task 5, but strongly recommended if you want to compare a statistical model against a deep learning approach on the same data and metrics.

6.1 Windowing the Series

Neural sequence models need fixed-length input windows. Convert the scaled series into (X, y) pairs: the previous n_steps readings predict the next one.

```
def make_windows(series, n_steps):
    X, y = [], []
    for i in range(len(series) - n_steps):
        X.append(series[i:i+n_steps])
        y.append(series[i+n_steps])
    return np.array(X), np.array(y)

n_steps = 48    # last 48 hours -> next hour
X_train, y_train = make_windows(train_scaled, n_steps)
X_test, y_test = make_windows(test_scaled, n_steps)
X_train = X_train.reshape(-1, n_steps, 1)
X_test = X_test.reshape(-1, n_steps, 1)
```

6.2 Simple RNN Baseline

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

rnn = Sequential([
    SimpleRNN(32, input_shape=(n_steps, 1)),
    Dense(1)
])
rnn.compile(optimizer='adam', loss='mse')
```

6.3 LSTM Model


```

lstm = Sequential([
    LSTM(64, return_sequences=True, input_shape=(n_steps, 1)),
    Dropout(0.2),
    LSTM(32),
    Dense(1)
])
lstm.compile(optimizer='adam', loss='mse')

es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
history = lstm.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=50, batch_size=32,
    callbacks=[es], verbose=1
)

```

6.4 Inverse-Transform & Evaluate

```

y_pred_scaled = lstm.predict(X_test)
y_pred = scaler.inverse_transform(y_pred_scaled)
y_true = scaler.inverse_transform(y_test.reshape(-1, 1))

mae_lstm = mean_absolute_error(y_true, y_pred)
rmse_lstm = np.sqrt(mean_squared_error(y_true, y_pred))

```

TIP — Common LSTM pitfalls

Fitting the scaler on the full dataset instead of train-only → leakage.
 Forgetting to inverse-transform before computing MAE/RMSE → metrics reported in scaled units, not °C.
 Too few epochs/patience for the LSTM to converge, then declaring 'ARIMA wins' unfairly — give the network a fair shot with early stopping and a validation split before comparing.

6.5 Training Curve & Comparison Plot

```

plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.legend(); plt.title('LSTM Training Curve'); plt.savefig('lstm_loss.png', dpi=150)

plt.figure(figsize=(11,4))
plt.plot(test.index[n_steps:], y_true, label='actual')
plt.plot(test.index[n_steps:], y_pred, label='LSTM forecast')
plt.plot(test.index, pred_mean, label='SARIMA forecast', alpha=0.7)
plt.legend(); plt.title('SARIMA vs LSTM'); plt.savefig('model_comparison.png', dpi=150)
)

```

7. Model Comparison & General Tips

7.1 Comparison Table Template

Trainees should fill this in with their own results:

Model	MAE	RMSE	MAPE (%)	Notes
ARIMA (non-seasonal)	—	—	—	Baseline, ignores daily cycle
SARIMA (m=24)	—	—	—	Captures daily seasonality
Simple RNN	—	—	—	Bonus
LSTM	—	—	—	Bonus

7.2 General Tips

- **Multi-step vs single-step:** decide upfront whether you're forecasting one point ahead or a full horizon (e.g. next 24h) recursively — the evaluation setup differs.
 - **Recursive forecasting drift:** when feeding predictions back as inputs for multi-step forecasts, errors accumulate — report error growth by horizon (see 5.3).
 - **Seasonal decomposition first:** run `seasonal_decompose` or STL before committing to ARIMA orders; it tells you if you need seasonal terms at all.
 - **Keep a naive baseline:** compare against a persistence model ("tomorrow = today") or seasonal-naive ("same hour yesterday") — if ARIMA/LSTM can't beat this, something's wrong.
 - **Don't over-difference:** differencing more than needed adds noise and can hurt forecasts; check ACF of residuals after each differencing step.
 - **Document assumptions:** note frequency, timezone handling, and any imputed gaps in your final notebook/report — reproducibility matters as much as accuracy.
-

8. Deliverables Checklist

- Cleaned, resampled, gap-free time series with documented preprocessing steps.
- Stationarity test results (ADF + KPSS) and differencing decisions, justified.
- ACF/PACF plots with your manually reasoned (p,d,q)(P,D,Q,m), plus the `auto_arma` result.
- Fitted SARIMA model with residual diagnostics (`plot_diagnostics` + Ljung-Box).
- Forecast plot with confidence intervals vs actual test data.
- MAE / RMSE / MAPE reported on the held-out test set, plus walk-forward validation scores.
- (Bonus) LSTM/RNN pipeline with training curve and side-by-side comparison against SARIMA.
- Short written conclusion: which model performed best, and why you think that is.